



Examen: Aplicación de Clima en Vivo con JavaScript

Objetivo del Examen

En este examen práctico deberás desarrollar una **aplicación web interactiva para la consulta del estado del tiempo en diversas ciudades**, conectando una interfaz frontend desarrollada con **HTML, CSS y JavaScript Vanilla** a un **servidor backend local en Node.js/Express**.

La aplicación debe permitir:

- 1. Consultar dinámicamente la lista de ciudades disponibles desde el backend.
- 2. Obtener y mostrar las condiciones meteorológicas detalladas (temperatura, condición, humedad, viento y sensación térmica).
- 3. Guardar el historial de consultas en el navegador mediante `localStorage`.
- 4. Mostrar el historial y permitir su limpieza interactiva.

Tabla de Entregas / Issues de GitHub

Cada entrega se corresponde con un **issue automático** en tu repositorio de GitHub. Para cerrar cada issue automáticamente, incluye el commit sugerido exacto al subir tu solución a la rama principal (`main`).

Entrega	Tarea a Realizar	Commit Sugerido
#1	Vincular <code>css/styles.css</code> y <code>js/script.js</code> en <code>index.html</code> .	<code>feat(html): vincular css y script js al html</code>
#2	Consumir la API local (<code>/api/ciudades</code> y <code>/api/clima/:id</code>) usando <code>fetch</code> y <code>async/await</code> .	<code>feat(js): consumir api de clima con fetch y async await</code>
#3	Renderizar dinámicamente las opciones del selector de ciudades y los datos climáticos en el DOM.	<code>feat(js): renderizar ciudades y datos de clima en el dom</code>
#4	Capturar el evento <code>submit</code> del formulario, prevenir la recarga con <code>preventDefault()</code> y procesar la consulta.	<code>feat(js): capturar submit del formulario y procesar consulta</code>
#5	Persistir las consultas en <code>localStorage</code> , renderizar el historial al iniciar y permitir su limpieza con <code>#btn-limpiar-historial</code> .	<code>feat(js): persistir y gestionar historial en localStorage</code>

Especificación Técnica y Requerimientos

1. Servidor Backend Local

El servidor Express provisto corre en el puerto `3000` con CORS habilitado:

- `GET http://localhost:3000/api/ciudades` : Devuelve el arreglo completo de ciudades con sus datos meteorológicos.

- `GET http://localhost:3000/api/clima/:id` : Devuelve los datos específicos de la ciudad solicitada (por ID o nombre).

Para iniciar el servidor backend:

```
npm start
```

2. Elementos Clave del DOM

- `#select-ciudad` : Elemento `<select>` donde deben cargarse las opciones (`<option value="id">Nombre, País</option>`).
- `#form-clima` : Formulario que dispara la consulta.
- `#clima-card` : Tarjeta donde se visualiza el clima actual (debe quitarse la clase `hidden` al consultar).
- `#ciudad-nombre` y `#ciudad-pais` : Nombre y país de la ciudad.
- `#temp-valor` : Temperatura en grados.
- `#condicion-texto` : Texto descriptivo del clima.
- `#icono-clima` : Emoji/icono del clima.
- `#sensacion-valor` , `#humedad-valor` , `#viento-valor` : Valores de los detalles técnicos.
- `#historial-lista` : Elemento `<ul>` donde se listan las consultas guardadas.
- `#btn-limpiar-historial` : Botón que limpia los datos en `localStorage` y actualiza la lista.

3. Almacenamiento Local (`localStorage`)

- **Clave obligatoria:** `'clima_historial'`
- **Estructura:** Arreglo de objetos con `{ ciudad, temperatura, condicion, fecha }` .
- Utilizar `JSON.stringify()` para guardar y `JSON.parse()` para leer.

Comandos de Prueba y Autoevaluación

Antes de entregar, podés autoevaluar tu trabajo localmente:

```
# Ejecutar todas las pruebas automáticas
npm test

# Ejecutar una prueba individual
npm run test:link
npm run test:fetch
npm run test:render
npm run test:events
npm run test:storage

# Validar estilo y calidad de código
npm run lint
npm run format:check
```

Instrucciones para la Ejecución Local

1. Instalar dependencias:

```
npm install
```

2. Iniciar el servidor local:

```
npm start
```

3. Abrir `index.html` en el navegador (usando la extensión **Live Server** de VS Code).
4. Abrir la consola de herramientas de desarrollador (**F12**) para verificar peticiones de red y depurar posibles errores.